AI with Rav

Learn AI the way you actually think.

DAY 3 of 30

PYTHON

Strings — working with text like clay

WHAT YOU'LL LEARN TODAY

- A string is a row of characters
- Slicing — pulling out a piece
- Joining strings with +
- f-strings (the easy way)
- Handy string tools

Strings — working with text like clay

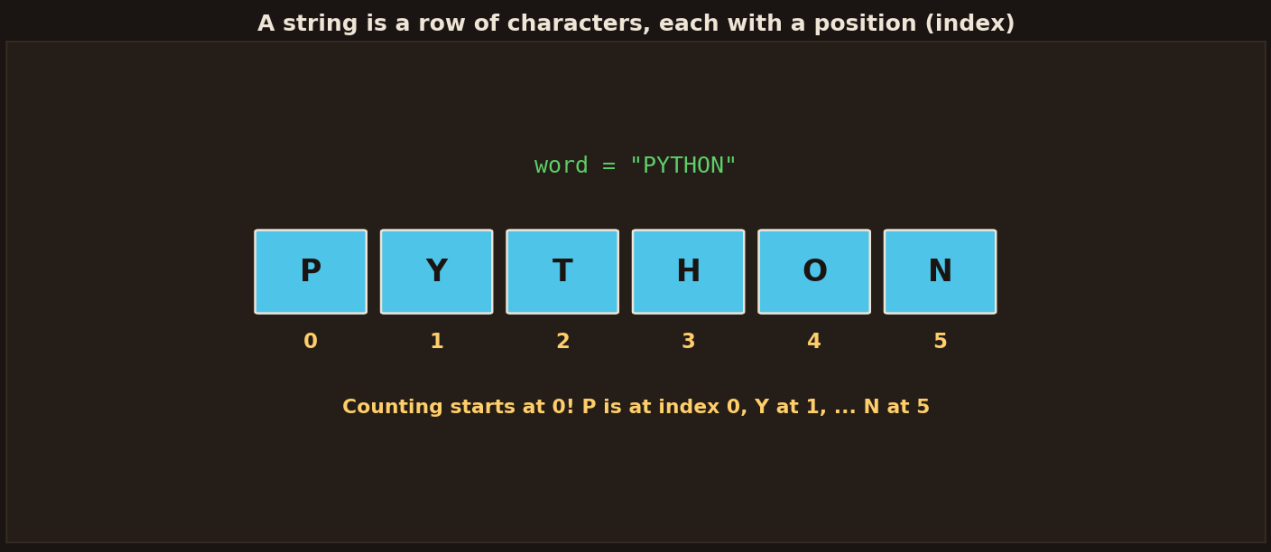
> In One Line: A string is just text — and Python lets you shape it like clay. Join words together, pull out a piece, change the case, or drop variables straight into a sentence. Text is everywhere in AI (reviews, chats, documents), so this is a skill you'll use constantly.

Start here: text is everywhere

You met strings on Day 2 — text wrapped in quotes, like `"Rav"`. But strings are far more than storage. In AI you'll work with text all the time: user reviews, chat messages, documents, names, labels. So Python gives you powerful, easy tools to shape text like a potter shapes clay.

A string can use single or double quotes — `'hi'` and `"hi"` are the same. Let's learn to work with them.

A string is a row of characters



The string "PYTHON" laid out as boxes: P, Y, T, H, O, N — each with a position number below (its index). Counting starts at 0, so P is at index 0 and N is at index 5.

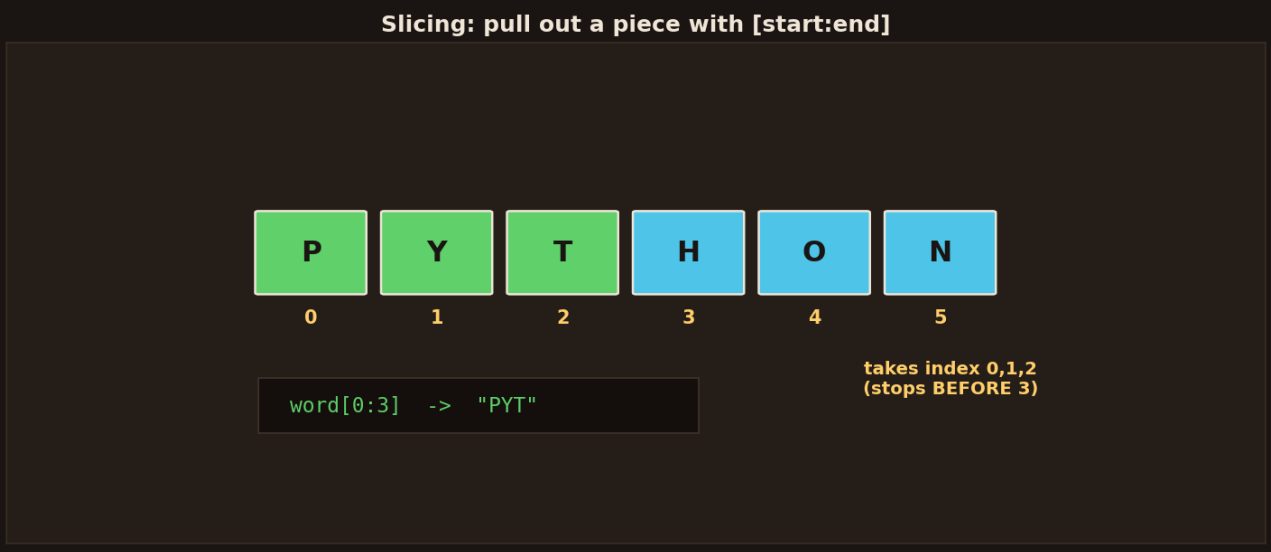
Under the hood, a string is just a **row of characters**, each with a **position** called an **index:**

- The first character is at index **0** (not 1 — this trips up every beginner!).
- So in `"PYTHON"`: P is 0, Y is 1, T is 2, H is 3, O is 4, N is 5.

```
word = "PYTHON"
print(word[0])     # -> P   (the first character)
print(word[3])     # -> H
```


> The #1 beginner surprise: **counting starts at 0, not 1.** So `word[0]` is the *first* letter. Almost every programming language does this. Burn it in: position 0 = first. Get this and half of "off by one" bugs disappear.

Slicing — pull out a piece



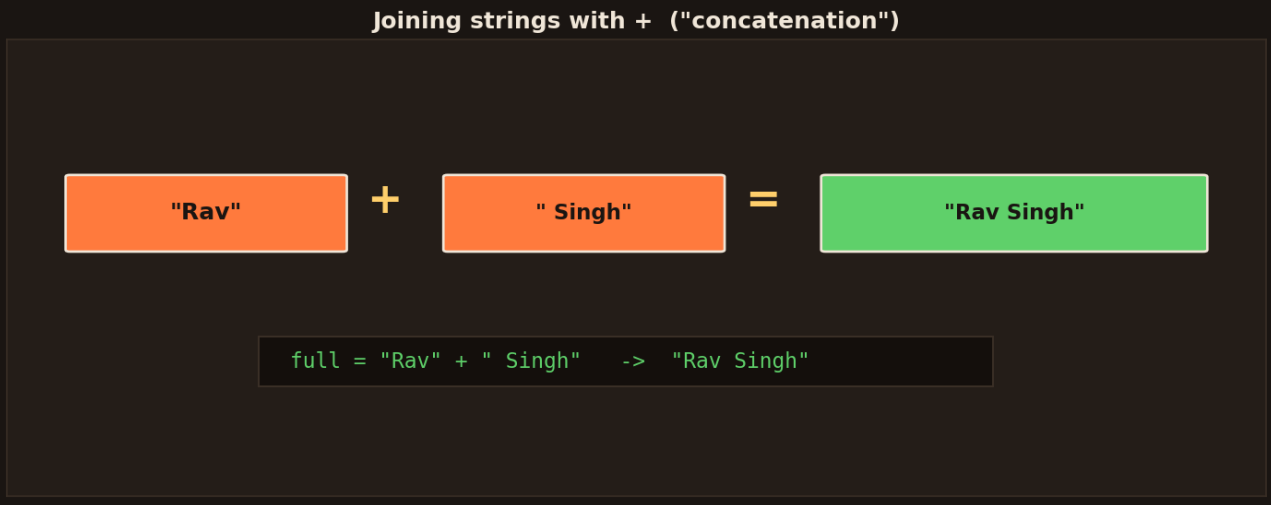
Slicing "PYTHON" with word[0:3] highlights P, Y, T (index 0, 1, 2) and gives "PYT". The slice stops BEFORE index 3 — so 3 is not included.

Want more than one character? **Slice** it with `[start:end]`:

```
word = "PYTHON"
print(word[0:3])    # -> "PYT"   (index 0, 1, 2)
print(word[2:])     # -> "THON"  (from 2 to the end)
print(word[:2])     # -> "PY"    (from start to before 2)
```

> The catch: the slice goes up to but **NOT including** the end number. `word[0:3]` gives indexes 0, 1, 2 — it stops *before* 3. Think "start, up to but not including end." Leave a side blank to mean "the start" or "the end." Slicing is everywhere in data work.

Joining strings with +



Joining strings: "Rav" + " Singh" glues them into "Rav Singh". The + sign sticks strings end to end (called concatenation).

Stick strings together with `+` (called **concatenation**):

```
first = "Rav"
last  = "Singh"
full  = first + " " + last     # note the space " " in the middle
print(full)                    # -> "Rav Singh"
```

> The `+` sign glues strings end to end. Notice we added `" "` (a space) between them — otherwise you'd get `"RavSingh"`. One thing to know: you can only `+` a string with a string. `"age " + 30` would error — you'd need to turn the number into text first (we'll see that soon). But there's an even nicer way to mix text and variables…

f-strings — the easy way

An f-string: f"Hi {name}, you are {age}" drops the variables name and age straight into the text. With name="Rav" and age=30, it becomes "Hi Rav, you are 30".

The cleanest way to build a message is an **f-string** — put an `f` before the quotes and drop variables inside `{}`:

```
name = "Rav"
age  = 30
print(f"Hi {name}, you are {age}")     # -> "Hi Rav, you are 30"
```

> This is the modern favourite. The `f` tells Python "fill in the blanks," and anything inside `{}` gets replaced by that variable's value — even numbers, automatically. No `+`, no fiddly spaces. You'll use f-strings constantly to print results, build labels, and log what your code is doing.

Handy string tools

Useful string methods: .upper() makes it UPPERCASE, .lower() lowercase, .strip() trims spaces, .replace() swaps text, and len() counts characters.

Strings come with built-in tools called **methods** — attach them with a dot:

- `.upper()` / `.lower()` → change the case: `"rav".upper()` → `"RAV"`.
- `.strip()` → trim spaces off the ends: `" rav ".strip()` → `"rav"`.
- `.replace("old", "new")` → swap text: `"cat".replace("c","b")` → `"bat"`.
- `len("Rav")` → count characters → `3`.

These come up daily when cleaning text data — real data is messy (extra spaces, wrong case), and `.strip()` + `.lower()` fix a huge amount of it. `len()` isn't a method (no dot) — it's a function you wrap around anything to count its length. You'll use it on strings, and later on lists too.

Recap — the 20-second version

- A string is a **row of characters**, positions starting at **index 0.**
- **Slice** a piece with `[start:end]` — it stops *before* the end number.
- **Join** strings with `+` (mind the spaces).
- **f-strings** — `f"Hi {name}"` — drop variables into text the easy way.
- Tools: `.upper()`, `.lower()`, `.strip()`, `.replace()`, and `len()` to count.

Next up — Video 4: Numbers & Math. Python is a powerful calculator built in. Next we do arithmetic, meet the difference between whole and decimal numbers, and learn the neat operators for "remainder" and "power". See you tomorrow.